



PC40 Lab 2 Report: From Sequential Neural-Network Training to Explicit Multithreading

Bora Yalçın

1 Overview

This report was written for the second PC40 lab, where a sequential neural network that trains on MNIST is turned into a multithreaded program. The important thing to say first is that this is not a machine learning lab. The network was given to us already working, and I was not asked to change the algorithm or improve the accuracy. My job was to understand where the independent work is, hand that work out to threads, keep the numerical behaviour the same, and then measure what I actually gained. I did the whole lab in C++ because the lab asks to pick one language and C++ is the one I am most comfortable with for this kind of low level work.

The path I followed is the one the lab sheet describes: sequential, then decomposition, then threads, then a race condition, then synchronization, then reduction, then performance. I kept every intermediate version as its own file instead of overwriting one file, so that I can still run the broken version and show what went wrong. All the numbers in this report come from runs on my own laptop and I say which session each one belongs to, because the baseline moves between sessions and mixing them would make the speedups meaningless.

2 Tools and Hardware Used

2.1 Hardware

- **CPU:** Intel Core Ultra 9 285H, 16 logical processors, 16 physical cores, 1 thread per core (no SMT)
- **GPU:** NVIDIA GeForce RTX 5070 Laptop GPU (8GB VRAM)
- **RAM:** 32GB DDR5
- **Caches:** L1d 480 KiB (12 instances), L1i 768 KiB (12 instances), L2 28 MiB (9 instances), L3 24 MiB (1 instance)

2.2 Hybrid Core Layout

This CPU is a hybrid design, so the 16 cores are not equal. I checked this with `lscpu -all -extended`:

Listing 1: `lscpu -all -extended` (trimmed to the relevant columns)

	CPU	NODE	SOCKET	CORE	L1d:L1i:L2:L3	ONLINE	MAXMHZ	MINMHZ
1								

2	0	0	0	0 0:0:0:0	yes	5400,0000	400,0000
3	1	0	0	1 4:4:1:0	yes	5400,0000	400,0000
4	2	0	0	2 16:16:4:0	yes	5400,0000	400,0000
5	3	0	0	3 20:20:5:0	yes	5400,0000	400,0000
6	4	0	0	4 24:24:6:0	yes	5400,0000	400,0000
7	5	0	0	5 28:28:7:0	yes	5400,0000	400,0000
8	6	0	0	6 2:0	yes	4500,0000	400,0000
9	7	0	0	7 18:18:2:0	yes	4500,0000	400,0000
10	8	0	0	8 2:0	yes	4500,0000	400,0000
11	9	0	0	9 22:22:2:0	yes	4500,0000	400,0000
12	10	0	0	10 3:0	yes	4500,0000	400,0000
13	11	0	0	11 26:26:3:0	yes	4500,0000	400,0000
14	12	0	0	12 3:0	yes	4500,0000	400,0000
15	13	0	0	13 30:30:3:0	yes	4500,0000	400,0000
16	14	0	0	14 64:64:8	yes	2500,0000	400,0000
17	15	0	0	15 66:66:8	yes	2500,0000	400,0000
18							

Reading the `MAXMHZ` column, the cores split into three groups:

- CPUs 0 to 5: 5400 MHz, so 6 performance cores (P cores), each with its own L2.
- CPUs 6 to 13: 4500 MHz, so 8 efficiency cores (E cores), in two clusters of four that share an L2 (the L2 index is 2 for CPUs 6 to 9 and 3 for CPUs 10 to 13).
- CPUs 14 and 15: 2500 MHz, so 2 low power E cores, and the last column shows they do not have the shared L3.

This matters a lot in Part J. There is no hyper-threading here, so 16 threads really do get 16 cores, but those cores are not the same speed. I come back to this when the speedup stops improving.

2.3 Operating System

- Kubuntu 26.04.1 LTS (Resolute Raccoon)
- Kernel: Linux 7.0.0-34-generic
- Desktop Environment: KDE Plasma 6.6.6 (Wayland)

2.4 Compiler and Flags

- GCC / G++ 15.2.0 (Ubuntu 15.2.0-16ubuntu1)
- Sequential baseline: `g++ -O2 -std=c++17 nn_sequential.cpp -o nn_sequential`
- Multithreaded versions: `g++ -O2 -std=c++17 -pthread ...`, except the pool version and the combined I/J/K binary which need `-std=c++20` for `std::barrier`
- ThreadSanitizer builds: `-O1 -g -fsanitize=thread -pthread`
- Python 3.14.4 (only used for the helper scripts, not for the implementation)

2.5 Workload

Every measurement in this report uses the same workload unless I say otherwise: 12000 training samples, 2000 test samples, 3 epochs, batch size 64, learning rate 0.08, and the same seed 42 for the weight initialisation. Data loading is not included in the training time.

2.6 My Files

- `nn_sequential.cpp`: the given reference, unchanged.
- `nn_multithreaded_D_naive.cpp`: Parts D and E, unsynchronized shared accumulation (deliberately unsafe).
- `nn_multithreaded_F_mutex.cpp`: Part F, one mutex around the whole `sample_grad()` call.
- `nn_multithreaded_G2_optimized.cpp`: Part G, private gradients per worker plus a reduction after `join()`.
- `nn_multithreaded_G_threadlocal.cpp`: the G variant where the buffers are allocated once and only zeroed per batch.
- `nn_multithreaded_K_pool.cpp`: Part K, persistent workers with two `std::barrier` objects.
- `nn_multithreaded_IJ.cpp`: the binary I used for Parts I, J and K, with `-mode create` (the G design), `-mode pool` (the K design), `-bs` and a weight checksum.
- `check_I.py`, `bench_J.py`, `plot_J.py`: the Part I check, the Part J benchmark and the plots.
- `false_sharing.cpp`: the optional HPC challenge.

3 Part A - Neural Networks in a Nutshell

3.1 Objective

Before writing any thread, I have to know what the program actually computes and where each piece of state lives. The point of this part is to locate the weights, the forward pass, the gradient computation, the mini-batch loop and the weight update in the given sequential file, so that in Part C I can decide what is safe to share.

3.2 Answers

The network is 784 inputs, then 64 hidden neurons with ReLU, then 10 outputs with softmax. It is trained with mini-batch SGD for 3 epochs, batch size 64 and learning rate 0.08, on 12000 training and 2000 test samples. The weights and biases are the members `W1`, `b1`, `W2` and `b2` of `struct NN`. Here are the regions the lab asks for:

The single most useful detail I found is in the signature of `sample_grad()`:

Listing 2: `nn_sequential.cpp`, the signature that decides everything

```
1 // Reads:  W1, b1, W2, b2 (the model)           -> shared, read-only
2 // Adds to: g1, gb1, g2, gb2 (gradient accumulators),
3 //          loss and correct (statistics)         -> whoever owns them
4 // The method is 'const': it never modifies the model.
5 void sample_grad(const float* x, int label,
6                 vector<float>& g1, vector<float>& gb1,
7                 vector<float>& g2, vector<float>& gb2,
8                 float& loss, int& correct) const;
9
```

Region	Function or place	What it does
[1] Data loading	<code>load_idx()</code> , <code>synthetic()</code>	Reads the MNIST IDX files into one flat pixel array and one label array, or builds fake data for a smoke test.
[2] Initialisation	<code>NN::NN()</code>	Fills <code>W1</code> and <code>W2</code> from a normal distribution with a fixed seed 42, so every run starts from the same model.
[3] Forward pass	first half of <code>NN::sample_grad()</code>	For one sample, computes the hidden layer with ReLU, then the logits, then the softmax probabilities.
[4] Backward pass	second half of <code>NN::sample_grad()</code>	For the same sample, computes the gradient contribution and adds it into the gradient accumulators it was given.
[5] Mini-batch loop	inner loop of <code>NN::train()</code>	Calls <code>sample_grad()</code> once per sample of the current batch, all into the same accumulators. This is the loop I parallelise.
[6] Weight update	end of each batch in <code>NN::train()</code>	Applies $W \leftarrow W - \eta G/n$ once, after the whole batch gradient is known.
[7] Evaluation	<code>NN::accuracy()</code>	Forward pass only, counts how many test samples are classified correctly.

Table 1: The seven regions of the sequential file

The method is `const`, so it can never write to the model. Everything it writes goes through the accumulator references the caller passes in. That means the decision about what is shared and what is private is made entirely by the caller, not inside the function. This is exactly what lets me parallelise the loop without touching the math.

3.3 Results

The mini-batch loop is the parallel target, and it is only three lines:

Listing 3: and [6]nn_sequential.cpp, region [5] and [6]

```

1 // Batch gradient accumulators, zeroed for every batch.
2 vector<float> g1(W1.size()), gb1(H), g2(W2.size()), gb2(C);
3
4 // ---- [5] Mini-batch accumulation ----
5 for (int i = s; i < e2; i++)
6     sample_grad(&d.x[(size_t)i * d.d], d.y[i], g1, gb1, g2, gb2, L, ok);
7
8 // ---- [6] Weight update: W <- W - lr * G / n ----
9 float a = lr / n;
10 for (size_t i = 0; i < W1.size(); i++) W1[i] -= a * g1[i];
11
```

Each iteration of that loop reads the same weights and adds into the same accumulators. Nothing in the loop body depends on the previous iteration, which is what makes the samples independent.

3.4 Conclusion

The structure is clean enough that the parallel opportunity is obvious once you look at it: the expensive part is one call per sample, and the samples of a batch do not talk to each other.

The only sequential dependency is between batches, because the update at the end of one batch changes the weights the next batch reads.

4 Part B - Establish the Sequential Baseline

4.1 Objective

Get a reference time and a reference set of numbers that every later version is compared against, and record the machine conditions that produced it.

4.2 Answers

The lab asks why a parallel program must be compared against a stable baseline. My answer is that speedup is a ratio, $S_P = T_1/T_P$, so a bad T_1 poisons every single number in the table. If the baseline run happened to be slow because the CPU was throttling or something else was running, then every speedup comes out too optimistic, and the conclusion I draw from the curve is wrong. It is not enough to be accurate, it also has to be fair: the dataset size, the epoch count, the batch size, the optimisation flags and the machine state have to be identical for the baseline and for every parallel configuration, otherwise I am measuring the difference between two experiments instead of the effect of threading.

I ran into this directly. The same sequential binary, unchanged, gave a median of 1.33097 s in my first session but 1.5062 s and 1.5565 s in the two later sessions where I did the Part J sweep. Nothing about the program changed. On a laptop the clock speed depends on turbo behaviour, temperature and the power profile, so the baseline drifts between sessions. Because of this I only compare numbers measured inside the same session, and I never build a speedup out of two different sessions. I say which session every table belongs to.

4.3 Results

Three runs of the sequential version, same conditions:

Run	Training time (s)
1	1.31162
2	1.33097
3	1.34761
Median T_{seq}	1.33097

Table 2: Sequential baseline, session 1, -O2 -std=c++17

Listing 4: ./nn_sequential ../data, session 1

```

1 Epoch 1 loss=1.04301 train_acc=74.9583%
2 Epoch 2 loss=0.407451 train_acc=88.7833%
3 Epoch 3 loss=0.335226 train_acc=90.5083%
4 Test accuracy: 86.95%
5
```

These four lines are the reference. From here on, any version that does not reproduce loss 1.04301 at epoch 1 and 86.95% test accuracy is broken, and I check that before I look at its time.

4.4 Conclusion

The baseline is a pair, not a single number: a time and a set of expected outputs. The time tells me whether a change helped, and the outputs tell me whether I am still allowed to look at the

time at all. The session drift also taught me to record when a measurement was taken, which turned out to be the most useful habit of the whole lab.

5 Part C - Find the Parallelism Before Writing Threads

5.1 Objective

Classify every piece of state as shared read-only, thread-private, or shared writable before writing any thread, and work out how to split a batch between workers.

5.2 Answers

Here is my classification.

Data	Class	Why
Weights and biases w_1, b_1, w_2, b_2	shared read-only	<code>sample_grad()</code> is <code>const</code> , so during the batch nobody writes them.
Input samples x	shared read-only	Each worker only reads its own rows, and nothing writes the dataset.
Labels y	shared read-only	Same reason as the samples.
Local gradients	thread-private	One accumulator per worker, only that worker touches it.
Temporaries in <code>sample_grad</code>	thread-private	<code>h</code> , <code>logit</code> , <code>prob</code> , <code>dl</code> , <code>dh</code> are declared inside the function, so each call gets its own.
Global gradient g_1, gb_1, g_2, gb_2	shared writable	Every worker's contribution ends up here, so it needs either a lock or a reduction.
Loss and correct counters	shared writable	Same problem as the gradient if workers add into them directly.
Updated weights	shared writable	They are written in the update step, which must happen after all workers finish, so they are never written during the computation.

Table 3: Part C classification

I want to be honest about this part, because my first attempt had two mistakes. I first wrote that the global gradient was read-only, which is clearly wrong, it is the one thing every worker wants to write. And I put the loss and accuracy counters in the thread-private column, which is only true in the design I ended up with in Part G, not in the naive design of Part D where they are shared. Getting these two wrong is exactly what produced the race in Part E, so the table above is the corrected version and I think making the mistake first is the reason I now understand the difference.

For batch size 64 and $P = 4$, with the batch starting at sample s , each worker gets a contiguous quarter:

The general method, which also works when the batch is not divisible by P : let $base = n/P$ and $rem = n \bmod P$ with integer division. The first rem workers get $base + 1$ samples and the rest get $base$ samples. The ranges stay contiguous, with no gaps and no overlaps, and the largest and smallest chunk differ by at most one sample.

This is not a theoretical case here, it actually happens on every epoch. With 12000 samples and batch size 64 we get $12000 = 187 \times 64 + 32$, so the last batch of each epoch has only 32 samples, which is 8 per worker at $P = 4$. If I had written a naive split that assumes divisibility, the last batch would silently drop or duplicate samples and the loss would not match the baseline.

Worker	Sample interval
T_0	$[s, s + 16)$
T_1	$[s + 16, s + 32)$
T_2	$[s + 32, s + 48)$
T_3	$[s + 48, s + 64)$

Table 4: Sample intervals for batch size 64 and $P = 4$

5.3 Results

This is the splitting code I use in the final version:

Listing 5: `nn_multithreaded_IJ.cpp`, `split()`

```

1 // Split [s, e2) into P contiguous chunks (first 'rem' chunks get +1).
2 static void split(int s, int e2, int P, vector<int>& a, vector<int>& b) {
3     int n = e2 - s, base = n / P, rem = n % P, cur = s;
4     for (int p = 0; p < P; p++) {
5         int len = base + (p < rem ? 1 : 0);
6         a[p] = cur; b[p] = cur + len; cur += len;
7     }
8 }
9

```

5.4 Conclusion

Doing this on paper before writing threads was the most useful step of the lab. The classification table is basically the design document: everything in the shared writable column is a place where I will need synchronization, and the whole point of Part G is to make that column as small as possible. My two mistakes also show that the classification is not obvious, and that getting it wrong has a direct, reproducible consequence.

6 Part D - First Explicit Multithreaded Implementation

6.1 Objective

Write the first threaded version using the create all, start all, join all pattern, with a configurable worker count, and check it at $P = 1$ before worrying about concurrency.

6.2 Answers

I copied the sequential file into `nn_multithreaded_D_naive.cpp` and replaced the inner sample loop with P workers, each handling a contiguous chunk of the batch. Everything else is unchanged: same seed, same dataset size, same batch size, same learning rate, same epochs.

In this first version I deliberately did the simplest possible thing and let all workers write straight into the same shared accumulators. I knew this was wrong, that is the whole point, because Part E asks me to expose the race rather than avoid it.

The lab warns not to start a worker and immediately join it in the same loop. If I did that, worker 0 would be created, run to completion and be joined before worker 1 is even created, so the program would be sequential with extra overhead. So I push all the threads into a vector first, and only join them in a second loop.

6.3 Results

Listing 6: nn_multithreaded_D_naive.cpp, the unsafe worker loop

```
1 vector<std::thread> workers;
2 int start = s;
3 for (int p = 0; p < P; p++) {
4     int count = base + (p < remainder ? 1 : 0);
5     int end = start + count;
6
7     // UNSAFE: g1, gb1, g2, gb2, L, ok are shared and written
8     // by every worker with no synchronization whatsoever.
9     workers.emplace_back([this, &d, start, end, &g1, &gb1, &g2, &gb2, &L, &ok]() {
10         for (int i = start; i < end; i++) {
11             sample_grad(&d.x[(size_t)i * d.d], d.y[i], g1, gb1, g2, gb2, L, ok);
12         }
13     });
14
15     start = end;
16 }
17
18 for (auto& t : workers) t.join();
19
```

At $P = 1$ the output matched the sequential baseline exactly:

Listing 7: ./nn_D_naive ../data -threads 1

```
1 Workers P = 1
2 Epoch 1 loss=1.04301 train_acc=74.9583%
3 Epoch 2 loss=0.407451 train_acc=88.7833%
4 Epoch 3 loss=0.335226 train_acc=90.5083%
5 Test accuracy: 86.95%
6
```

6.4 Conclusion

The $P = 1$ check is worth more than it looks. With one worker there is no concurrency at all, so if the numbers match the baseline I know my restructuring is correct: the chunk split covers every sample exactly once, the accumulators are zeroed at the right moment, and the update still happens once per batch. Anything that breaks after that can only be a concurrency problem. That separation saved me a lot of guessing in the next part.

7 Part E - Deliberately Expose a Race Condition

7.1 Objective

Run the unsafe version with several workers, show that the results change from run to run, and back that up with a tool instead of with anecdotes.

7.2 Answers

The accumulation $x += a$ looks like one operation in C++ but it is really three: read x from memory, add a , write the result back. Two threads can read the same old value, each add their own contribution to it, and each write back. One of the two updates is then simply gone. With 50176 floats in $g1$ alone and 64 samples per batch, there are a lot of chances for this to happen.

The more important point is that in C++ this is not just a lost update, it is a data race, and a data race is undefined behaviour. The standard gives no guarantee at all about what the program does. So I cannot reason about it as if it were just slightly inaccurate arithmetic.

This is also why one correct looking run proves nothing. Whether two threads interleave badly depends on timing, on how the OS schedules them, and on which core they land on. A run that gives the right answer only means that this particular interleaving happened to be harmless. A timing dependent bug is not disproved by a run that passes, which is why the sanitizer report matters more than the output.

7.3 Results

Three runs of the unsafe version at $P = 4$, nothing changed between them:

Run	Epoch 1 loss	Epoch 1 train acc (%)	Test acc (%)
1	1.04354	74.325	86.90
2	1.04206	74.6167	86.95
3	1.03729	74.45	87.00
Sequential	1.04301	74.9583	86.95

Table 5: The same program, the same input, three different answers (session 1)

Three runs, three different losses and three different accuracies. The sequential version gives exactly the same numbers every time, so this spread comes purely from the missing synchronization.

The time was about 0.84 s, which makes this the fastest version in the whole lab. It is also the only one that is wrong. This is the lab's point about a fast wrong program not being a successful parallel program, and it is easy to see why it is fastest: it does no synchronization at all, so it pays no cost for it.

I then built the same file with ThreadSanitizer and ran it on the small synthetic workload:

Listing 8: Building and running the sanitizer on the unsafe version

```

1 g++ -std=c++17 -O1 -g -fsanitize=thread -pthread \
2   nn_multithreaded_D_naive.cpp -o nn_tsan
3 ./nn_tsan --synthetic --limit 500 --threads 4
4
```

It reported 5 data races, and every one of them is a variable I put in the shared writable row of my Part C table:

Racing write	Line in <code>nn_multithreaded_D_naive.cpp</code>	Elements
<code>loss -= ...</code> (the shared L)	123	1
<code>correct += ...</code> (the shared ok)	124	1
<code>g2[k * C + c] += ...</code>	130	640
<code>gb2[c] += ...</code>	131	10
<code>g1[j * H + k] += ...</code>	140	50176

Table 6: The five data races reported by ThreadSanitizer

After fixing the design (private accumulators plus a reduction, which is Part G), the lab asks to run the sanitizer again. I did this on the final combined binary, in both modes:

Listing 9: ThreadSanitizer on the corrected version, both designs

```

1 $ g++ -std=c++20 -O1 -g -fsanitize=thread -pthread \
2     nn_multithreaded_IJ.cpp -o nn_tsan_fixed
3
4 $ ./nn_tsan_fixed --synthetic --threads 4 --mode create
5 Threads: 4 mode: create batch: 64
6 Epoch 1 loss=1.94510114 train_acc=85.45%
7 Epoch 2 loss=0.787674367 train_acc=100%
8 Epoch 3 loss=0.165782481 train_acc=100%
9 Test accuracy: 100%
10 Checksum: sum=104.595416212262 abs_sum=2148.36970187395
11 Training time: 6.1839665 s
12
13 $ ./nn_tsan_fixed --synthetic --threads 4 --mode pool
14 Threads: 4 mode: pool batch: 64
15 Epoch 1 loss=1.94510114 train_acc=85.45%
16 Epoch 2 loss=0.787674367 train_acc=100%
17 Epoch 3 loss=0.165782481 train_acc=100%
18 Test accuracy: 100%
19 Checksum: sum=104.595416212262 abs_sum=2148.36970187395
20 Training time: 4.32862132 s
21

```

No warnings at all in either mode, which is the output I wanted. The 100% accuracy here is not suspicious, it is just the synthetic smoke test data, which is much easier than MNIST. The times are also several times slower than normal because ThreadSanitizer instruments every memory access.

7.4 Conclusion

This part changed how I think about testing concurrent code. With sequential code I test by running it and looking at the answer. Here the answer looked almost right in all three runs, and someone who only ran it once would have shipped it. The sanitizer found the exact five variables I had already predicted on paper in Part C, which was a good feeling, and it gives a clean before and after: five races on the naive version, zero on the corrected one.

8 Part F - Correctness with Mutual Exclusion

8.1 Objective

Make the program correct using a lock, measure what that costs, and understand why a badly placed lock destroys the parallelism.

8.2 Answers

The invariant the lock protects is that a read-modify-write on the shared accumulators must complete without another thread touching the same memory in between. In other words, while one worker is between reading `g1[i]` and writing back the new value, no other worker may read or write `g1[i]`, and the same for `g2`, `gb1`, `gb2`, `L` and `ok`.

In my version I put a single `std::lock_guard` around the whole `sample_grad()` call. That is deliberately the coarse choice, because the lab asks what happens if the lock covers the entire sample computation.

The answer is that it destroys the parallelism completely. `sample_grad()` is where all the expensive work is, roughly 100 thousand multiply-adds per sample. If that whole call is inside the critical section, then only one worker can be doing useful work at any instant. The threads

do not overlap at all, they just take turns. So I get the sequential execution time plus the cost of locking and unlocking once per sample, plus the cost of the threads waiting.

The opposite extreme, taking the lock for every individual scalar update, is just as bad but for a different reason. There are 50890 gradient values per sample, so that would be about 50890 lock and unlock pairs per sample instead of one. Each one is cheap on its own, but they are contended, and the total locking overhead ends up far larger than the arithmetic it protects. Both extremes are wrong, and the useful lesson is that the size of the critical section is the thing you actually tune, not the presence of the lock.

8.3 Results

Listing 10: nn_multithreaded_F_mutex.cpp, the coarse lock

```

1  std::mutex accMutex;
2  ...
3  workers.emplace_back([this, &d, start, end, &g1, &gb1, &g2, &gb2, &L, &ok, &
    accMutex]() {
4      for (int i = start; i < end; i++) {
5          std::lock_guard<std::mutex> lock(accMutex);
6          sample_grad(&d.x[(size_t)i * d.d], d.y[i], g1, gb1, g2, gb2, L, ok);
7      }
8  });
9

```

Version	Time (s)	Correct?
Sequential baseline	1.33097	yes (reference)
Part E unsafe, $P = 4$	≈ 0.84	no, different every run
Part F mutex, $P = 4$, run 1	2.13158	yes
Part F mutex, $P = 4$, run 2	2.04828	yes

Table 7: Correct but slower than doing nothing in parallel at all (session 1)

At $P = 4$ the mutex version gives results identical to $P = 1$, so it is genuinely correct. It is also about 1.6 times slower than the sequential baseline of the same session, which means adding three extra threads made the program worse.

8.4 Conclusion

This is the version that made the trade-off concrete for me. The unsafe version was fast and wrong, this one is correct and slow, and neither is a successful parallel program. What I take from it is that a mutex does not remove a race, it removes the concurrency that caused the race, and if the critical section contains the useful work then that is the same thing as not parallelising at all. The fix is not a better lock, it is a design where the workers have nothing to fight over.

9 Part G - Better Design: Thread-Local Gradients

9.1 Objective

Get rid of the shared writes during the expensive computation, by giving each worker its own private accumulator and combining them afterwards.

9.2 Answers

Each worker p now owns `local_g1[p]`, `local_gb1[p]`, `local_g2[p]`, `local_gb2[p]`, plus its own `local_loss[p]` and `local_correct[p]`. During the whole forward and backward computation no worker writes to memory that another worker touches. The weights are read only, the dataset is read only, and the accumulators are private. After all workers are joined, the main thread sums the P private copies into one batch gradient and does one weight update.

The main computation no longer needs a mutex, and that is not an optimisation, it is a consequence of the design. There is no shared writable state left during the computation phase, so there is nothing to protect.

The lab also asks about the memory cost. The network has $784 \times 64 + 64 + 64 \times 10 + 10 = 50890$ trainable parameters, so one float32 gradient copy is about 199 KiB. With $P = 16$ that is about 3.1 MiB of extra memory. On a machine with 32 GB of RAM this is nothing, and it is far smaller than the L3 cache, so I did not treat it as a trade-off at all here.

It would change for a bigger model. With 10^8 parameters one gradient copy is about 400 MB, and with $P = 64$ that is about 25 GB of private accumulators. At that point the private copy design is simply not affordable and you would need something else, like partial reductions in a tree so that not all P copies exist at once, or workers sharing an accumulator per group instead of per thread. For this network none of that is justified, and inventing a trade-off that does not exist at this size would be dishonest.

9.3 Results

Listing 11: Part G, each worker writes only to its own slot

```
1  workers.emplace_back([this, &d, start, end, p,  
2                          &local_g1, &local_gb1, &local_g2, &local_gb2,  
3                          &local_loss, &local_correct]() {  
4      for (int i = start; i < end; i++) {  
5          sample_grad(&d.x[(size_t)i * d.d], d.y[i],  
6                      local_g1[p], local_gb1[p], local_g2[p], local_gb2[p],  
7                      local_loss[p], local_correct[p]);  
8      }  
9  });  
10  
11 for (auto& t : workers) t.join();  
12  
13 // ---- Reduction (single-threaded, race-free by construction) ----  
14 for (int p = 0; p < P; p++) {  
15     for (size_t i = 0; i < g1.size(); i++) g1[i] += local_g1[p][i];  
16     ...  
17     L += local_loss[p];  
18     ok += local_correct[p];  
19 }  
20
```

At $P = 4$ the results match $P = 1$, so the design is correct. The times were 1.25871 s, 1.44726 s and 1.40639 s, so a median of 1.40639 s against a sequential baseline of 1.33097 s in that session. Correct, but barely faster, and on the median actually slightly slower.

That surprised me, so I made a hypothesis and tested it. My guess was that allocating P fresh vectors of about 199 KiB for every mini-batch was the dominant cost. I wrote a variant where the accumulators are allocated once before the batch loop and only zeroed with `std::fill` per batch:

Listing 12: The G variant, allocate once and only zero per batch

```

1 // Allocated ONCE for the whole training run.
2 vector<vector<float>> local_g1(P, vector<float>(W1.size()));
3 ...
4 // Zero out (not reallocate) before every mini-batch.
5 for (int p = 0; p < P; p++) {
6     std::fill(local_g1[p].begin(), local_g1[p].end(), 0.0f);
7     ...
8 }
9

```

Version	Run 1	Run 2	Run 3	Median
G, allocate per batch	1.25871	1.44726	1.40639	1.40639
G variant, allocate once	1.39860	1.37515	1.38777	1.38777

Table 8: Testing the allocation hypothesis at $P = 4$, times in seconds (session 1)

The two are the same within the run to run noise, so my hypothesis was wrong. The allocation was not the problem. What is left as the real per batch cost is thread creation and joining, which happens $188 \times 3 = 564$ times per run at batch size 64 (12000 divided by 64 is 187.5, so 187 full batches plus one partial batch of 32 samples), and the serial reduction that the main thread does alone after every one of those batches.

9.4 Conclusion

This is the design the rest of the lab is built on. It removes the mutex not by locking better but by making sure there is nothing shared to lock. The disproved hypothesis was almost as useful as the design itself, because it pointed me at the two costs that actually matter, per batch thread management and the serial reduction, and those are exactly the two things Parts J and K go after.

10 Part H - Synchronization and Reduction

10.1 Objective

Make the reduction explicit, do exactly one weight update per batch, and understand what a missing synchronization point would break.

10.2 Answers

The reduction combines the P private gradients into one batch gradient, and in the same phase it also combines the private loss and correct counters. Then there is exactly one weight update for the batch. I do the reduction in a fixed order, $p = 0$ to $P - 1$, which matters in Part I: floating point addition is not associative, so a fixed order makes the result reproducible for a given P .

The synchronization point is `join()` in the create-per-batch design and a `std::barrier` in the pool design, which is exactly the split the lab sheet describes.

The lab then asks what could happen if one worker started the next batch with the new weights while another worker was still computing the previous batch with the old ones. Two things go wrong at the same time. First, it is a data race in the plain C++ sense: the main thread would be writing `W1` and `W2` while a worker is reading them, which is undefined behaviour. Second, even ignoring that, the worker could read a mix of old and new weights, because the update loop walks through 50176 values and the worker could read some before and some after they were touched. Its gradient would then belong to no real model state at all, it would be the

gradient of a model that never existed. The batch gradient G would end up mixing contributions computed from different weight versions. In practice that means a non-deterministic loss and accuracy, and if the updates are large enough it can make the training unstable rather than just noisy.

My two barriers in the pool version prevent both directions of this. `sync_done` guarantees that nobody is still computing when the main thread starts the update, and `sync_start` guarantees that nobody starts the next batch before the update has finished.

10.3 Results

Listing 13: `nn_multithreaded_IJ.cpp`, `reduce_and_update()`

```

1 // [6] Reduction (fixed order p = 0..P-1) + exactly ONE weight update.
2 std::fill(g1.begin(), g1.end(), 0.0f);
3 ...
4 for (int p = 0; p < P; p++) {
5     for (size_t i = 0; i < g1.size(); i++) g1[i] += lg1[p][i];
6     for (int i = 0; i < H; i++)           gb1[i] += lgb1[p][i];
7     for (size_t i = 0; i < g2.size(); i++) g2[i] += lg2[p][i];
8     for (int i = 0; i < C; i++)           gb2[i] += lgb2[p][i];
9     L += lloss[p];
10    ok += lok[p];
11 }
12 float a = lr / n;
13 for (size_t i = 0; i < W1.size(); i++) W1[i] -= a * g1[i];
14 for (int i = 0; i < H; i++)           b1[i] -= a * gb1[i];
15 for (size_t i = 0; i < W2.size(); i++) W2[i] -= a * g2[i];
16 for (int i = 0; i < C; i++)           b2[i] -= a * gb2[i];
17

```

The checkpoint the lab asks for is satisfied by construction. The weights are read only while workers compute, the workers write only private accumulators, and the update happens only after every worker has finished. There is no window in which a shared value is written by one thread and read by another.

10.4 Conclusion

The thing I find neat about this design is that correctness is not enforced by a lock anywhere, it comes out of the phase structure. Compute, then synchronize, then reduce and update, and repeat. Once the phases are clean there is no critical section left to protect, and the only remaining question is how much the phase boundary itself costs, which is what Part J measures.

11 Part I - Correctness Validation

11.1 Objective

Show that changing P does not change what the program computes, define tolerances that are strict enough to catch a real bug but loose enough to accept floating point reordering, and explain the differences that remain.

11.2 Answers

Bit for bit identity is not expected, because a parallel reduction adds the same numbers in a different order. Floating point addition is not associative, so $(a + b) + c$ and $a + (b + c)$ can differ

in the last bits. With $P = 2$ the batch gradient is built as two partial sums that are then added, with $P = 4$ as four, and those groupings are simply not the same arithmetic.

What makes the effect visible in the weights rather than in the loss is that the tiny difference is not just made once. It goes into the weight update, and the next batch computes its gradient from those slightly different weights, and so on for all 564 updates of a run. So the difference accumulates in the weights while the loss stays almost identical.

I used these tolerances:

- Epoch loss: relative difference $\leq 10^{-4}$.
- Test accuracy: absolute difference ≤ 0.1 percentage points, which is 2 images out of the 2000 test images, so it allows a couple of borderline samples to flip without allowing a real degradation.
- Checksum: relative difference $\leq 10^{-4}$, but normalised by $\Sigma|w|$ rather than by Σw .

That last point needs an explanation because it is the one thing I changed after looking at the first results. The checksum Σw is about 141 while $\Sigma|w|$ is about 2263, because the weights are roughly half positive and half negative and they cancel each other out. If I divide the absolute difference by Σw I am dividing by a number that is 16 times smaller than the actual size of the data, so the relative difference comes out about 16 times too pessimistic. In my runs that plain version gives 8.9×10^{-5} , which sits right at the 10^{-4} limit and looks alarming even though nothing is wrong. Normalising by $\Sigma|w|$ instead gives about 5.6×10^{-6} , which is a much more honest measure of how much the weight vector actually moved.

The reason the tolerances work is the gap between the two effects. The reordering noise is around 10^{-6} , while the Part E race moved the loss by around 10^{-3} . That is a factor of about a thousand, so a threshold in between cleanly separates harmless reordering from a real bug.

11.3 Results

Listing 14: python3 check_I.py

```

1 mode = create
2
3 P |      loss ep1      loss ep2      loss ep3 | test acc |      checksum sum |      checksum |w|
4 -----
5 1 | 1.043010950 0.407450527 0.335225761 | 86.95% | 141.285307878773 | 2263.480919742520
6 2 | 1.043010830 0.407450557 0.335226148 | 86.95% | 141.296993006917 | 2263.479802912010
7 4 | 1.043010120 0.407450348 0.335225821 | 86.95% | 141.297878702553 | 2263.480661437070
8
9 Differences vs P=1:
10 P | max rel loss | acc diff (pp) |      rel sum |      rel |w| | verdict
11 -----
12 2 | 1.154e-06 | 0.00 | 8.271e-05 | 4.934e-07 | PASS
13 4 | 7.958e-07 | 0.00 | 8.897e-05 | 1.141e-07 | PASS
14
15 Tolerances: loss rel <= 0.0001, test acc <= 0.1 pp, checksum rel <= 0.0001
16 ALL PASS
17

```

Running the same check with `-mode pool` produced a bit identical table, every digit the same. That is expected, because the two designs use the same chunk split and the same fixed reduction order, so they perform exactly the same additions in exactly the same order. Only the way the threads are managed differs, and that does not touch the arithmetic.

The normalised checksum difference, $\Delta \Sigma w / \Sigma|w|$, works out to 5.16×10^{-6} for $P = 2$ and 5.55×10^{-6} for $P = 4$. I computed these by hand from the checksum columns above, because the version of `check_I.py` that produced this output still prints the plain `rel sum` column.

Summarising the differences against $P = 1$:

The test accuracy is 86.95% for $P = 1, 2$ and 4, identical to the sequential baseline.

Quantity	$P = 2$	$P = 4$	Tolerance
Max relative loss difference	1.15×10^{-6}	7.96×10^{-7}	10^{-4}
Test accuracy difference (pp)	0.00	0.00	0.1
Relative difference of $\Sigma w $	4.93×10^{-7}	1.14×10^{-7}	10^{-4}
$\Delta\Sigma w/\Sigma w $	5.16×10^{-6}	5.55×10^{-6}	10^{-4}

Table 9: Part I, every quantity is two to three orders of magnitude inside its tolerance

11.4 Conclusion

The validation is what lets me talk about performance at all. Every difference is around 10^{-6} or 10^{-7} while the tolerance is 10^{-4} , and the race in Part E was around 10^{-3} , so the check would have caught it easily. The part I learned the most from is the normalisation: my first checksum metric was technically a relative difference but it measured the wrong thing, and it would have made me chase a bug that did not exist.

12 Part J - Granularity and Scalability

12.1 Objective

Measure execution time against thread count, compute speedup and efficiency, and explain why the curve does not follow the ideal $S_P = P$ line.

12.2 Answers

As the lab asks, T_1 here is the one worker run of the same parallel structure, not the original sequential program. That gives the cleanest scaling study, because it compares like with like. The original sequential baseline for this session was 1.5565 s and I report it separately rather than folding it into the speedups.

I measured $P = 1, 2, 4, 8, 16$, three runs each, and took the median. The machine has 16 physical cores with no SMT, so $P = 16$ is meaningful rather than oversubscription. I did both designs, create-per-batch (the Part G structure) and the persistent pool (the Part K structure), and I also swept the batch size over 32, 64, 128 and 256. The main study stays at batch size 64 as the lab asks.

12.3 Results

P	create per batch (G)			persistent pool (K)		
	T_P (s)	S_P	E_P	T_P (s)	S_P	E_P
1	1.8038	1.00	1.00	1.8883	1.00	1.00
2	1.1317	1.59	0.80	1.3556	1.39	0.70
4	0.9627	1.87	0.47	1.1771	1.60	0.40
8	1.1146	1.62	0.20	1.0591	1.78	0.22
16	1.2371	1.46	0.09	1.1395	1.66	0.10

Table 10: Main scalability study, batch size 64, median of 3 runs. Original sequential baseline of the same session: 1.5565 s

The required table from Section 18 of the lab sheet, using the create structure and with the test accuracy measured separately with `nn_IJ`:

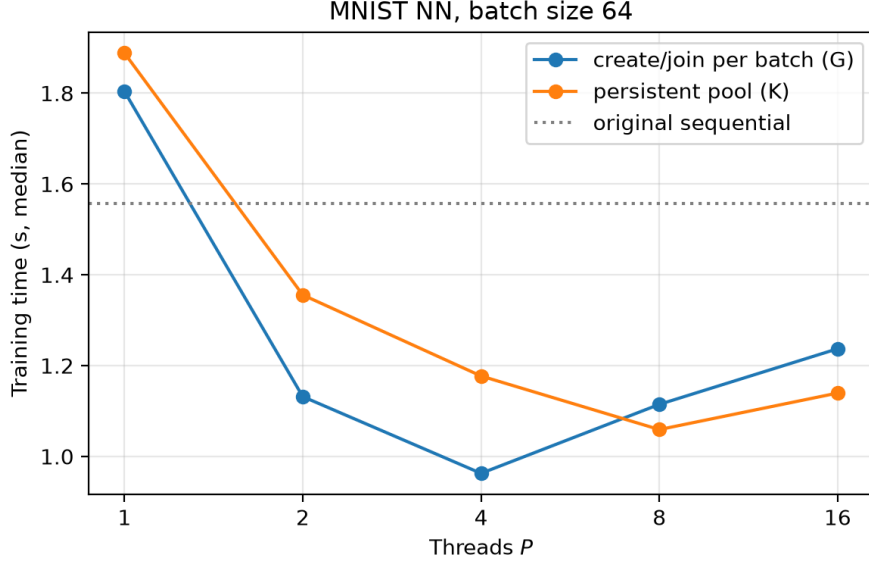


Figure 1: Execution time versus thread count, batch size 64

Threads P	Median T_P (s)	Speedup S_P	Efficiency E_P	Test accuracy (%)
1	1.8038	1.00	1.00	86.95
2	1.1317	1.59	0.80	86.95
4	0.9627	1.87	0.47	86.95
8	1.1146	1.62	0.20	86.95
16	1.2371	1.46	0.09	86.95

Table 11: The required experimental table (create per batch, batch size 64)

The accuracy column is the same in every row, which is the point: adding threads changes the time and nothing else.

An earlier session where I only measured batch size 64 gave the same shape, with the create version best at $P = 4$ at 0.9291 s and a speedup of 1.78. The absolute numbers moved a little between sessions but the curve looks the same, so I am confident the shape is real and not a one off.

12.4 Discussion

Thread management overhead. The clearest evidence is the first row of the table. The create version at $P = 1$ takes 1.8038 s while the plain sequential program in the same session takes 1.5565 s. That is about 0.25 s of pure overhead before any threading benefit exists, just from creating and joining one thread per batch and going through the private buffer and reduction machinery. At $P = 16$ the program creates and destroys $16 \times 564 \approx 9000$ threads over a run, and each one costs a system call and some scheduling work.

Synchronization. Every batch has a synchronization point: one `join()` in the create design, or two barrier arrivals in the pool design. The number of these per run depends only on how many batches there are, which is why it drops so fast as the batch grows:

The right hand column is the uncomfortable one. At batch size 32 with 16 threads, each worker does 2 samples and then waits at a barrier. The synchronization costs more than the work.

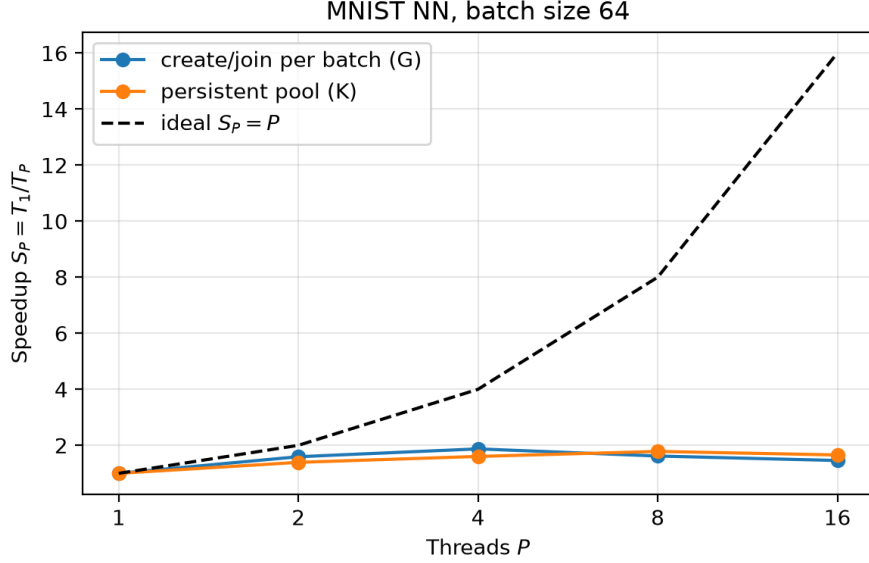


Figure 2: Speedup versus thread count, batch size 64, with the ideal $S_P = P$ line

Batch size	Batches per run (3 epochs)	Samples per thread at $P = 16$
32	1125	2
64	564	4
128	282	8
256	141	16

Table 12: Synchronization points per run, and how little work each thread gets

Reduction cost and the serial fraction. This is the real limit. After every batch the main thread alone sums P buffers of about 51 thousand floats each, zeroes the target buffers and applies the update. That work grows with P while the parallel work per thread shrinks, so the serial part of the program gets worse exactly when I add more threads. Fitting Amdahl’s law,

$$S_P = \frac{1}{f + (1 - f)/P},$$

to my measurements gives $f \approx 0.38$ at batch size 64 with the create design at $P = 4$, and $f \approx 0.30$ at batch size 256 with the pool design at $P = 16$. The interesting thing is that f is not constant. At batch size 32 the fitted f climbs from about 0.42 at $P = 2$ to about 0.97 at $P = 16$, while at batch size 256 it stays around 0.28 to 0.36. If the serial fraction were a fixed property of the algorithm it would not move like that. It moves because the overhead is paid per batch, so the more batches I have and the less work each one contains, the larger a share the serial part takes.

Granularity. This is the knob that actually helped. The per batch serial cost does not depend on the batch size, but the parallel work does, it is proportional to the number of samples in the batch. So a bigger batch amortises the same overhead over more useful work. The effect is exactly what I measured:

Two things move together: the speedup goes up from 1.43 to 2.89, and the P where the peak happens moves from 4 to 16. The best result overall is the pool design at batch size 256 with 16 threads, 0.603 s, which is 2.58 times faster than the original sequential baseline of that session. At the other end, batch size 32 with 16 threads in the create design takes 1.89 s, which is slower than just running the sequential program.

Batch size	Best create	Best pool	Batches per run
32	1.43 at $P = 4$	1.52 at $P = 4$ and $P = 8$	1125
64	1.87 at $P = 4$	1.78 at $P = 8$	564
128	2.04 at $P = 8$	2.42 at $P = 16$	282
256	2.18 at $P = 16$	2.89 at $P = 16$, 0.603 s	141

Table 13: Best speedup per batch size. The peak moves to the right and gets higher

I have to add a caveat here, because this is not a free performance knob. Changing the batch size changes the training itself: a bigger batch means fewer weight updates per epoch, so the loss and accuracy curves are not the same. I am comparing implementations of the same algorithm only within one batch size. That is why the main study stays at 64.

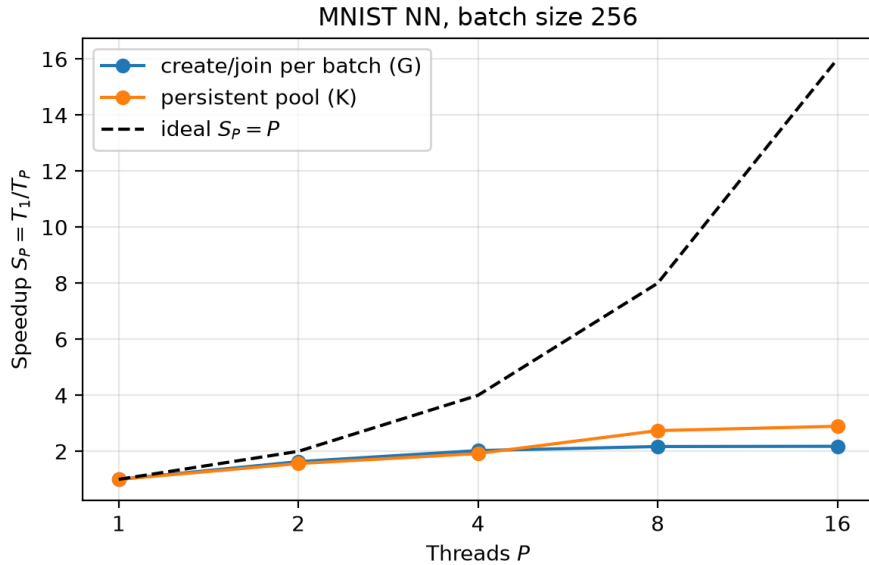


Figure 3: Speedup at batch size 256, where the pool design reaches 2.89 at $P = 16$

Load balance, physical versus logical cores. My chunks are balanced to within one sample, so on a uniform CPU the load would be even. This CPU is not uniform. There is no SMT, so 16 threads really do get 16 distinct cores, but 6 of them run at 5.4 GHz, 8 at 4.5 GHz and 2 at 2.5 GHz. Equal chunks on unequal cores means unequal finishing times, and a join or a barrier waits for the slowest one. Once I go past about 6 threads some of the work lands on the slower cores, so the batch takes as long as the slowest chunk no matter how fast the P cores finished. This is part of why the curve flattens after $P = 4$ to $P = 8$.

Memory and cache. Each worker’s `local_g1` is about 200 KB, so at $P = 16$ there is around 3.2 MB of private accumulators live at once. Writing them is fine, each worker touches only its own. The reduction is the awkward part, because the main thread has to read 16 buffers that were just written by 16 different cores. Those lines are dirty in other cores’ caches, so the reduction generates cache coherence traffic and misses rather than running out of local cache.

12.5 Conclusion

The speedup is real but modest, 1.87 at the main configuration and 2.89 at the best one, on a machine with 16 cores. The limit is not the parallel part, which scales fine, it is everything

around it: a serial reduction after every batch whose cost grows with P , a synchronization point per batch, and cores that are not equal. Granularity was the only lever that moved the peak meaningfully, and even that runs out because I cannot grow the batch without changing the training.

13 Part K - Worker Reuse

13.1 Objective

Test whether creating the threads once and reusing them for every batch is worth the extra complexity, compared to creating them per batch.

13.2 Answers

The create design makes and destroys P threads for every mini-batch, 564 times per run. The pool design creates P workers once and coordinates them with two barriers. As the lab asks, I used an established primitive, `std::barrier` from C++20, and not a home made flag barrier. The `stop` flag exists but it is only ever read right after a barrier, so its visibility is guaranteed by the barrier itself and it is not a synchronization mechanism of its own.

Two barriers are needed, not one. `sync_start` releases the workers once the main thread has handed out the ranges for the new batch, and `sync_done` tells the main thread that every worker has finished so it can reduce and update. Both are constructed with $P + 1$ participants, because the main thread takes part in both.

13.3 Results

Listing 15: `nn_multithreaded_K_pool.cpp`, the persistent worker

```
1 auto worker_fn = [&](int p) {
2     while (true) {
3         sync_start.arrive_and_wait();    // wait for main to hand out work (or stop)
4         if (stop) return;
5
6         std::fill(local_g1[p].begin(), local_g1[p].end(), 0.0f);
7         ...
8         for (int i = w_start[p]; i < w_end[p]; i++) {
9             sample_grad(&d.x[(size_t)i * d.d], d.y[i],
10                        local_g1[p], local_gb1[p], local_g2[p], local_gb2[p],
11                        local_loss[p], local_correct[p]);
12         }
13
14         sync_done.arrive_and_wait();    // tell main this worker is finished
15     }
16 };
17
```

The shutdown needs care, because the workers are blocked on `sync_start` and will never notice a flag on their own:

Listing 16: Shutting the pool down

```
1 // Release workers one final time so they see stop == true
2 stop = true;
3 sync_start.arrive_and_wait();
4 for (auto& t : workers) t.join();
5
```

In an earlier session with the separate files, at $P = 4$ and batch size 64, the G version had a median of 1.40639 s and the pool version gave 1.33573 s, 1.2386 s and 1.20098 s, a median of 1.2386 s. Correct in both cases, matching $P = 1$.

From the full sweep, same session for both columns:

Configuration	create (s)	pool (s)	Verdict
bs 64, best of each	0.963 ($P=4$)	1.059 ($P=8$)	no gain, create is 10% faster
bs 64, $P = 16$	1.237	1.140	pool 8% faster
bs 128, $P = 16$	0.881	0.720	pool 18% faster
bs 256, $P = 16$	0.750	0.603	pool 20% faster
bs 32, $P = 1$	1.937	2.233	pool 15% slower

Table 14: Create per batch versus persistent pool

The pattern is consistent. Where thread management is a big share of the total time, which means many threads and enough work per batch to keep them busy, the pool wins by 8 to 20 percent. Where it is not, the pool’s two barrier arrivals per batch cost more than the thread creation it saves, and it loses.

I also ran a core placement check at $P = 1$ and batch size 64, three interleaved rounds of each design:

Placement	create (s)	pool (s)
P cores, <code>taskset -c 0-5</code>	3.83, 3.78, 3.77	2.01, 2.05, 1.92
E cores, <code>taskset -c 6-13</code>	1.85, 1.76, 1.77	1.75, 1.67, 1.70
Unpinned	1.92, 1.82, 1.87	1.83, 1.86, 1.81

Table 15: Core placement at $P = 1$, batch size 64

I report this as an observation only. On this hybrid CPU the $P = 1$ timings depend strongly on where the threads are placed, and the small difference between create and pool at $P = 1$ is scheduler dependent. I am not going to offer an explanation for it, because I did not measure anything that would let me test one.

13.4 Conclusion

Worker reuse is worth it, but only at scale. At my main configuration, batch size 64, it gives no measurable gain, and it costs real complexity: two barriers, a stop flag, a shutdown protocol that has to release the workers one last time, and lambdas capturing by reference that must not outlive the data they point at. The honest summary is that neither design removes the actual bottleneck. Both still do a serial reduction after every batch, and that is why even the best configuration only reaches 2.89 on 16 cores.

14 Optional HPC Challenge - False Sharing

14.1 Objective

Show that variables which are logically independent can still slow each other down if they land on the same cache line.

14.2 Answers

Cache coherence works on whole cache lines, 64 bytes on this CPU, not on individual variables. If two threads write to different variables that happen to sit on the same line, the hardware still

has to keep that line coherent, so every write by one core invalidates the copy in the other core's cache. The line then bounces between cores. Nothing is logically shared, but the performance behaves as if it were.

My test has P threads, each incrementing only its own counter, 100 million times, through a `volatile` pointer so the compiler cannot keep it in a register. The only difference between the two variants is the layout:

Listing 17: `false_sharing.cpp`, the only difference is the alignment

```

1 struct Compact { long v = 0; };           // 8 bytes -> 8 counters per cache
   line
2 struct alignas(64) Padded { long v = 0; }; // 64 bytes -> 1 counter per cache
   line
3

```

14.3 Results

P	compact (3 runs, s)	padded (3 runs, s)
4	0.0460, 0.0322, 0.0339	0.0310, 0.0301, 0.0279
8	0.2600, 0.2596, 0.2618	0.0467, 0.0324, 0.0325

Table 16: False sharing, same work, different memory layout

At $P = 8$ the compact version is about 8 times slower. Eight counters of 8 bytes each is exactly 64 bytes, so all 8 land on one cache line and every single increment by any thread invalidates it for the other seven. At $P = 4$ the effect is small in this test, and I am only reporting the value rather than speculating about why.

The link back to the neural network is worth making. My `local_loss[p]` and `local_correct[p]` arrays have exactly the compact layout, so in principle they can false share. In practice it does not matter, because the whole batch only produces 64 such writes in total, spread over the workers, and each one sits next to about 100 thousand floating point operations. The traffic is negligible compared to the work. The gradient buffers are separate allocations of about 200 KB each, so they are nowhere near the same line. False sharing is simply not this program's bottleneck. The serial reduction is.

14.4 Conclusion

This was the experiment that surprised me most, because an 8 times slowdown from nothing but a memory layout is not something you would ever guess from reading the code. It also taught me the opposite lesson at the same time: the effect is real and dramatic in a microbenchmark that does nothing but increment, and completely irrelevant in my actual program. Knowing which one you are looking at is the whole skill.

15 Final Synthesis Questions

Q1. *Why was the mini-batch a natural unit for exposing parallelism?*

Because inside one batch the samples are independent. Every sample reads the same weights, which do not change during the batch, and produces its own gradient contribution. Nothing a sample computes is needed by another sample. The dependency only appears between batches, when the update changes the weights that the next batch will read.

Q2. *Why are shared data not automatically dangerous? Give a read-only shared example from this lab.*

Sharing is only dangerous when at least one thread writes. If everybody reads, there is nothing to interleave and no way to observe a half finished value. In my program the weights `W1`, `b1`, `W2`, `b2` and the dataset `x` and `y` are shared by all P workers during the whole computation phase, with no protection at all, and that is perfectly safe because `sample_grad()` is `const` and the update only happens after every worker has finished.

Q3. *Why is `x += a` on shared state a potential race?*

Because it is three operations, not one: read `x`, add `a`, write back. Two threads can both read the same old value and both write their own result, so one contribution is lost. I saw exactly this in Part E, where `ThreadSanitizer` reported five such races on the shared gradient arrays and the two counters, and three identical runs gave three different losses.

Q4. *Why can a mutex make a program correct but slow?*

Because the lock removes the race by removing the concurrency that caused it. In my Part F version the lock covers the whole `sample_grad()` call, so only one worker does useful work at a time. It gave results identical to $P = 1$, so it is correct, and it took about 2.1 s against a sequential baseline of 1.33 s, so four threads made it 1.6 times slower than one.

Q5. *Why do thread-local gradients reduce contention?*

Because there is nothing left to contend for. Each worker writes only to `local_g1[p]` and its other private slots, which no other thread touches, so during the expensive part there are no shared writes at all and no lock is needed. The shared writes are pushed into one short reduction phase at the end, done by a single thread, where contention is impossible by construction.

Q6. *Why is synchronization required before updating weights?*

Because the update writes the weights that the workers read. If the main thread started updating while a worker was still computing, that would be a data race and undefined behaviour, and the worker could read a mix of old and new values and produce a gradient for a model state that never existed. My barriers enforce both directions: `sync_done` means nobody is still computing when the update starts, and `sync_start` means nobody starts the next batch before the update has finished.

Q7. *What is a reduction in this application?*

It is combining the P private results into one. Concretely, summing the P local gradient arrays element by element into the batch gradient, and in the same pass summing the P local loss values and correct counts into the epoch totals. I do it in a fixed order $p = 0$ to $P - 1$ so that the result is reproducible for a given P , since floating point addition is not associative.

Q8. *Why does speedup normally deviate from $S_P = P$?*

Because $S_P = P$ assumes the only thing the program does is the parallel work. In my measurements four things get in the way. Thread management: the create version at $P = 1$ already costs 1.80 s against 1.56 s sequential. Synchronization: one join or two barrier arrivals per batch, 564 batches per run. The serial reduction, whose cost grows with P because the main thread has to sum P buffers, which is the main reason the fitted Amdahl fraction is around 0.30 to 0.38. And load imbalance, because equal chunks on unequal cores mean the barrier waits for the slowest.

Q9. *Why can using more threads than useful hardware contexts hurt performance?*

Beyond the number of hardware contexts the threads have to time share one core, so they do not run at the same time, and the operating system adds context switches and cache pollution on top. On my machine there are 16 contexts, so $P = 16$ is not oversubscription, but I still lose, for two other reasons. The contexts are not equal, 6 fast cores, 8 slower ones and 2 much slower ones, so the barrier waits for the slowest. And the work per thread gets too small: at batch size 32 with 16 threads each worker handles 2 samples, and that version is actually slower than the sequential program.

Q10. *Which parts of your manual solution do you expect OpenMP to simplify?*

Most of the plumbing. A single `#pragma omp parallel for reduction(+:...)` over the batch loop would replace my manual thread creation, my `split()` function, the per worker private buffers, the reduction loop and the barrier and pool machinery, because OpenMP keeps a thread team alive between parallel regions instead of making me build one. What it would not replace is the thinking: I would still have to decide what is shared and what is private, and the reduction would still be serial work that limits the speedup. I would also want to try `schedule(dynamic)`, because my fixed equal chunks are exactly what hurts on this hybrid CPU, and letting faster cores pull more chunks could help the imbalance.

16 Conclusion

The sentence the lab ends with is that parallel programming is not just adding threads, and going through these parts in order is what made me believe it rather than just agree with it.

My path through the lab was basically four versions of the same program. The naive one was the fastest thing I wrote all lab, about 0.84 s against a 1.33 s baseline, and it was wrong, giving a different answer on every run. The mutex version was correct and took 2.1 s, which is slower than not using threads at all. The thread-local version with a reduction was the first one that was both correct and faster, and it got there not by locking better but by making sure there was nothing shared to lock. The worker pool helped on top of that, but only when there were enough threads and enough work per batch to make thread creation worth avoiding.

The best result I got was 2.89 times faster on 16 cores, and I think understanding why it is not 16 is more valuable than the number itself. The parallel part scales fine. What does not scale is everything around it: a serial reduction after every batch whose cost grows with the number of threads, a synchronization point per batch, and 16 cores that are not the same speed. Granularity was the only real lever, and even that ran out, because I cannot keep growing the batch without changing the training itself.

The other thing I am taking away is about method rather than performance. Checking $P = 1$ first, keeping the broken versions instead of overwriting them, running a sanitizer instead of trusting three runs that looked fine, and writing down which session a measurement came from. None of those are about threads specifically, but without them I would have measured the wrong things and believed the results.

GitHub Repository

The full source code for this lab, including every intermediate version, the benchmark scripts and the plots, is available at: <https://github.com/BoraYalcinn/Multithreading-Multicore-Architecture>